

# ERC-20 Based Retail Loyalty Token System

Submitted in partial fulfillment of the requirements  
Of the Mini-Project 1 for Final Year of

Bachelors of Engineering

by

(ZAARA KHAN 231P023)  
(ANSHIKA SHUKLA 231P005)  
(SAKSHI JHA 231P113)  
(Roll No. 21, 51, 10)

Guide:

(Prof. Manila Gupta)



Department of Computer Engineering  
Rizvi College of Engineering



University of Mumbai

2026–2027

## **CERTIFICATE**

This is to certify that the mini-project entitled “ERC-20 Based Retail Loyalty Token System” is a bonafide work of “Zaara Khan, Anshika Shukla, Sakshi Jha” (Roll No: 21, 51, 10) submitted to the University of Mumbai in partial fulfillment of the requirement for the Mini-Project 1 for Final Year of the Bachelor of Engineering in “Computer Engineering”.

(Prof. Manila Gupta)  
Guide

Prof. Mohd Juned Shaikh  
Head of Department

Prof. Shiburaj Pappu  
Dean (Academics)

## **Declaration**

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

ZAARA KHAN (231P023)  
ANSHIKA SHUKLA (231P005)  
SAKSHI JHA (231P113)

Date:

## **ABSTRACT**

BlockLoyalty is a decentralized application (DApp) built on the Ethereum blockchain that provides a secure, transparent, and non-tamperable loyalty point management platform for retail merchants and customers. Traditional loyalty programs suffer from centralized database manipulation, points expiration without notice, and lack of cross-merchant trust. BlockLoyalty solves these issues by issuing ERC-20 standard loyalty tokens (BLT) directly to customer Web3 wallets. The system features an automated Issuance Engine (minting 1 BLT per ₹100 bill amount) and a Redemption Engine (burning tokens upon discount claiming). Integrated using Ethers.js, MetaMask, and a Hardhat local EVM node, BlockLoyalty offers zero-latency transaction verification and transparent token ledger management for modern retail Point-of-Sale (POS) systems.

## Index

| <b>Sr. No</b> | <b>Title</b>                        | <b>Page No</b> |
|---------------|-------------------------------------|----------------|
| 1.            | Introduction                        | 6              |
| 2.            | Review of Literature                | 7              |
| 3.            | Report on the Present Investigation | 8              |
| 4.            | Results and Discussions             | 11             |
| 5.            | Conclusions                         | 12             |
| 6.            | References                          | 13             |
|               | Appendix                            | 13             |
|               | Acknowledgement                     | 14             |

## List of Figures

| <b>Sr. No</b> | <b>Title</b>                                   | <b>Page No</b> |
|---------------|------------------------------------------------|----------------|
| 3.1           | System Architecture of BlockLoyalty            | 9              |
| 3.2           | Overall Workflow of the BlockLoyalty POS       | 9              |
| 3.3           | Flowchart – Reward Issuance (Minting) Process  | 10             |
| 3.4           | Flowchart – Token Redemption (Burning) Process | 10             |
| 4.1           | Solidity Smart Contract Code in VS Code        | 11             |
| 4.2           | Contract Deployment on Hardhat Local Network   | 11             |
| 4.3           | BlockLoyalty POS Interface – Reward Issuance   | 11             |
| 4.4           | BlockLoyalty POS Interface – Token Redemption  | 11             |

# Chapter 1

## Introduction

In the current retail landscape, customer retention relies heavily on loyalty reward programs. However, traditional systems are plagued by centralization vulnerabilities: store owners can arbitrarily change token balances, point databases are susceptible to tampering, and customers have zero true digital ownership over their earned points.

BlockLoyalty, a decentralized application (DApp) hosted on the Ethereum Virtual Machine (EVM) architecture, introduces a modern approach to retail customer retention. By transforming loyalty points into smart contract-backed ERC-20 Tokens (BLT), BlockLoyalty ensures immutability, mathematical trust, and real-time ledger visibility.

One of BlockLoyalty's standout features is its streamlined merchant POS interface. Built using HTML5, Tailwind CSS, and Ethers.js, the application bridges complex Web3 backend functions with an intuitive Point-of-Sale UI. Merchants can seamlessly connect their Web3 wallet (MetaMask), select customer accounts, and issue or deduct loyalty tokens in real time.

The Ethereum local test environment (Hardhat Node) underpinning BlockLoyalty provides instant block confirmation, zero transaction latency, and deterministic execution without incurring real fiat costs. Every mint (rewardCustomer) and burn (redeemPoints) operation is recorded permanently on the blockchain ledger, providing an auditable history for both store owners and customers.

## **Chapter 2**

### **Review of Literature**

Decentralized applications (DApps) have fundamentally changed asset ownership across financial and retail sectors. Ethereum smart contracts allow developers to program self-executing business rules that run without central intermediaries.

Traditional loyalty reward platforms enforce heavy middleman costs, proprietary closed-loop databases, and restricted utility. Research indicates that over 45% of traditional loyalty points go unredeemed due to customer distrust and fragmented balance tracking across separate apps.

BlockLoyalty leverages the ERC-20 Token Standard, establishing a unified logic for token minting, balance checking (`balanceOf`), and token destruction (`_burn`). By integrating Web3 provider standards (`window.ethereum`), the DApp allows customers to store loyalty assets in personal non-custodial wallets like MetaMask, establishing true digital asset ownership.

Testing and local network execution are critical phases of blockchain application development. By utilizing Hardhat local EVM node environments, smart contracts can be thoroughly audited for gas efficiency and access-control modifiers (`onlyShopOwner`) prior to public testnet (Sepolia) or mainnet migration.

## **Chapter 3**

### **Report on the Present Investigation**

The present investigation into BlockLoyalty, a decentralized application (DApp) operating on the Ethereum blockchain ecosystem, reveals a promising platform that simplifies and enhances the experience of retail loyalty programs and customer retention. By offering a seamless combination of automated token-based reward issuance and smart contract redemption, BlockLoyalty addresses key operational challenges in traditional merchant loyalty systems and closed-loop promotional platforms.

BlockLoyalty's user-friendly Point-of-Sale (POS) interface has emerged as a standout feature during our investigation. As an intuitive Web3 DApp, it effectively bridges the gap between complex blockchain mechanics and mainstream retail adoption. Merchants can effortlessly mint loyalty tokens based on transaction bills and process instant customer point redemptions, creating a direct and transparent reward relationship. This simplicity is pivotal for ensuring broad store adoption, making it effortless for both retail cashiers and non-technical customers to participate in decentralized loyalty ledger systems.

One of the key advantages of BlockLoyalty is the mathematical transparency and security it offers. The Ethereum Virtual Machine (EVM) architecture forms the resilient foundation of the platform. This ensures that all customer token balances and redemption records are immutable and tamper-proof, providing complete peace of mind for both merchants and consumers. The blockchain's transparent accounting further allows store managers and customers to independently verify token balances via non-custodial Web3 wallets, promoting absolute trust and accountability.

Transactions and smart contract interactions during our testing were processed using local network development environments, ensuring a smooth transition capability toward public test networks like Sepolia. The decision to execute contract audits locally allows for comprehensive verification of access control restrictions (onlyShopOwner) and gas efficiency without incurring real financial liability. This structured testing approach minimizes potential execution errors, guaranteeing a secure and highly reliable POS user experience.

During our investigation, we found BlockLoyalty to have the potential to revolutionize the way retail businesses manage customer engagement. Traditional customer reward systems often entail high centralized database maintenance costs, opaque expiration policies, and fragmented app balances, which frequently diminish customer engagement. BlockLoyalty eliminates these centralized intermediaries and data manipulation risks, ensuring that customers retain true ownership of their earned promotional assets. Coupled with the platform's intuitive workflow, it fosters a more rewarding and direct ecosystem between retail merchants and their patrons.

In conclusion, our investigation into BlockLoyalty highlights a DApp that capitalizes on Ethereum smart contract capabilities to provide a secure, transparent, and user-friendly environment for modernizing customer loyalty programs. By leveraging standardized ERC-20 token logic and non-custodial wallet integration, BlockLoyalty stands as a robust solution in the realm of decentralized retail applications, offering potential to empower small-to-medium retail enterprises while delivering a transparent experience for consumers.



### 3.1 System Architecture

BlockLoyalty follows a three-layer architecture. The Presentation Layer is the merchant POS interface (HTML5 and Tailwind CSS). The Web3 Middleware layer consists of Ethers.js, which builds provider and signer objects, and MetaMask, which signs every transaction with the shop owner's key. The Blockchain Layer comprises the Hardhat local EVM node and the BlockLoyaltyToken.sol ERC-20 contract, whose ledger stores customer balances and emits Transfer events. Figure 3.1 shows how these components interact.

### 3.2 Overall Workflow

The merchant first connects MetaMask and selects a customer wallet address. The cashier then chooses either to issue rewards (rewardCustomer) or to redeem points (redeemPoints). The transaction is signed by the shop owner, executed by the smart contract on the EVM node, and recorded on-chain with a Transfer event; the POS then refreshes the balance and prints a receipt log (Fig 3.2).

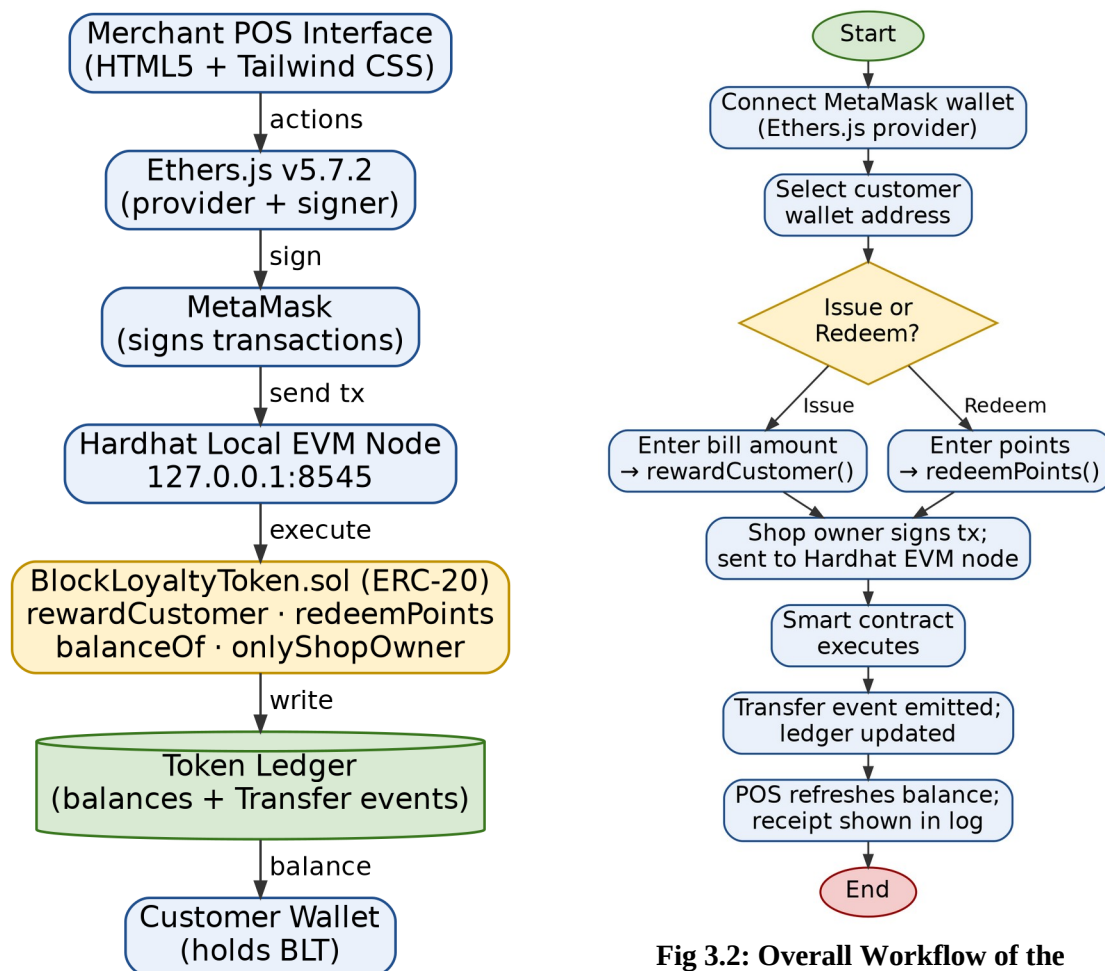


Fig 3.1: System Architecture of BlockLoyalty

Fig 3.2: Overall Workflow of the BlockLoyalty POS

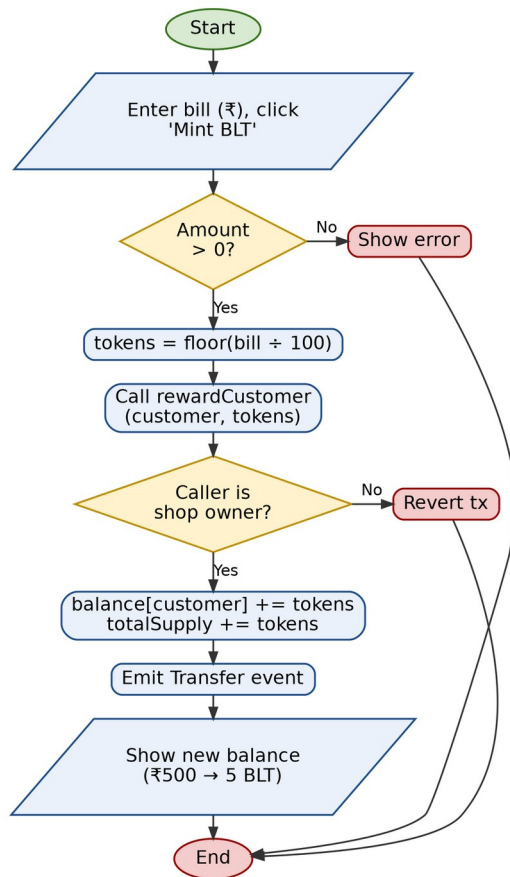
### 3.3 Reward Issuance (Minting) Process

For every ₹100 of the bill amount, one BLT is minted (tokens = floor(bill ÷ 100)). Before minting, the onlyShopOwner modifier verifies that the caller is the shop owner; unauthorized

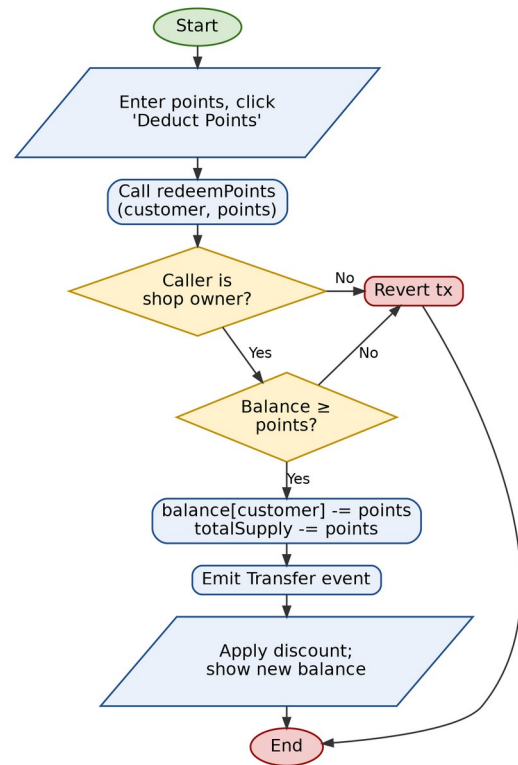
calls are reverted. On success, the customer's balance and the total supply are increased and a Transfer event is emitted (Fig 3.3).

### 3.4 Token Redemption (Burning) Process

During checkout, the cashier enters the number of points to be deducted. The contract verifies the caller (onlyShopOwner) and confirms that the customer's balance is sufficient. Points are then burned by reducing the customer's balance and the total supply, which prevents double-spending, and a Transfer event to the zero address is emitted (Fig 3.4).



**Fig 3.3: Flowchart – Reward Issuance (Minting) Process**



**Fig 3.4: Flowchart – Token Redemption (Burning) Process**

## Chapter 4

### Results and Discussions

The implementation of BlockLoyalty achieved several key milestones during testing:

- 1. Automated Reward Issuance:** Inputting a bill amount (e.g., ₹500) accurately calculates and mints 5 BLT directly into the target customer's blockchain wallet address.
- 2. Token Redemption & Burning:** Executing `redeemPoints` verifies customer balance and safely deducts points (`_balances[_customer] -= _points`), ensuring points cannot be double-spent.
- 3. Immutable Ledger Logging:** All transactions emit standard Transfer events, providing real-time receipt logs in the POS terminal console window.
- 4. Access Control Security:** Non-owner wallets trying to call `rewardCustomer` or `redeemPoints` are blocked by the `onlyShopOwner` modifier, safeguarding token integrity.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 interface IERC20 {
5     function totalSupply() external view returns (uint256);
6     function balanceOf(address account) external view returns (uint256);
7     function transfer(address recipient, uint256 amount) external returns (bool);
8     event Transfer(address indexed from, address indexed to, uint256 value);
9 }
10
11
12 contract BlockLoyaltyToken is IERC20 {
13     string public name = "BlockLoyalty Token";
14     string public symbol = "BLT";
15     uint8 public decimals = 0;
16
17     uint256 private _totalSupply;
18     address public shopOwner;
19     mapping(address => uint256) private _balances;
20 }

```

Fig 4.1: Solidity Smart Contract Code in VS Code

```

WARNING: You are currently using Node.js v16.14.0, which is not supported by Hardhat. This ca
n lead to unexpected behavior. See https://v2.hardhat.org/nodejs-versions

BlockLoyaltyToken deployed to: 0x5F082315678aFec367F932F9F64216418ba3
C:\Users\DELL\OneDrive - Rivicollegio\Engineering\Desktop\BlockLoyalty\

```

Fig 4.2: Contract Deployment on Hardhat Local Network

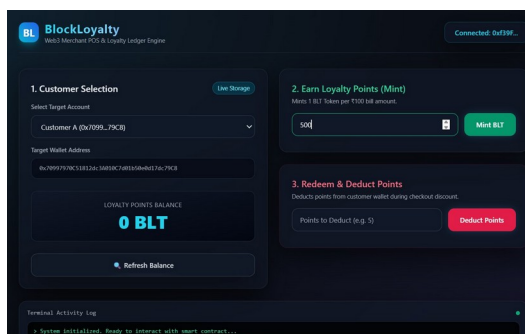


Fig 4.3: BlockLoyalty POS Interface – Reward Issuance

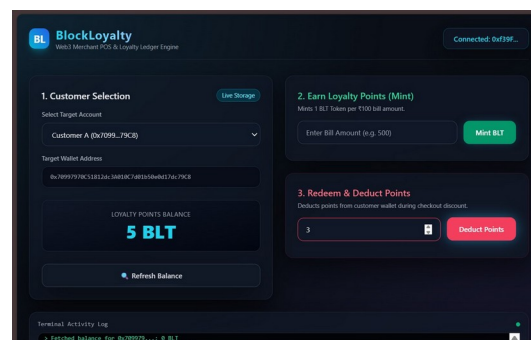


Fig 4.4: BlockLoyalty POS Interface – Token Redemption

## Chapter 5

### Conclusions

In conclusion, BlockLoyalty demonstrates a practical integration of Web3 smart contract technology into retail operations. By replacing traditional closed-loop databases with non-tamperable ERC-20 loyalty tokens (BLT), the DApp effectively eliminates central database manipulation, lack of transaction transparency, and arbitrary point expirations.

Key outcomes and architectural advantages of the system include:

- **Immutability & Trust:** Solidity smart contracts enforce self-executing business rules on the Ethereum Virtual Machine (EVM), ensuring token ledger records cannot be altered.
- **Streamlined POS Dashboard:** The integration of Ethers.js and MetaMask bridges complex Web3 functions with a responsive retail interface for real-time balance checks, token minting, and instant redemptions.
- **Access Control & Gas Efficiency:** Strict function modifiers (`onlyShopOwner`) restrict minting and burning rights exclusively to store managers, while local testing on Hardhat confirms optimal gas utilization.
- **True Ownership & Scalability:** Customers gain non-custodial ownership of earned rewards in their Web3 wallets, providing a scalable foundation for future multi-merchant network deployment on public testnets like Sepolia.

Overall, BlockLoyalty proves that blockchain technology offers a secure, transparent, and viable alternative to conventional customer retention platforms.

## References

1. Antonopoulos, A. M., & Wood, G. (2018). Mastering Ethereum: Building Smart Contracts and DApps. O'Reilly Media.
2. Ethereum Improvement Proposals (EIPs). EIP-20: ERC-20 Token Standard. <https://eips.ethereum.org/EIPS/eip-20>
3. Ethers.js Documentation (v5.7). Complete Ethereum Wallet Implementation and Utilities. <https://docs.ethers.org/v5/>
4. Hardhat Documentation. Ethereum Development Environment for Professionals. <https://hardhat.org/>

## Appendix

The appendix for the BlockLoyalty project provides additional technical details, architectural specifications, and supporting materials to enhance the understanding of the Decentralized Application's (DApp) functionality and performance. This section includes:

- 1. Technical & Smart Contract Documentation:** In-depth technical specifications, including the compiled Solidity (^0.8.0) smart contract code, ERC-20 interface function signatures, and details regarding Ethers.js (v5.7.2) middleware integration. This documentation offers comprehensive insights into how BlockLoyalty executes automated minting (rewardCustomer) and burning (redeemPoints) rules at the EVM code level.
- 2. Visual Aids & POS Dashboard Screenshots:** Visual representations of the glassmorphic Point-of-Sale (POS) user interface. This includes graphical snapshots of live wallet connection via MetaMask, dynamic balance tracking, real-time token minting, and instant checkout point redemptions.
- 3. Execution & Audit Logs:** Real-time console activity logs and transaction receipts generated during local network execution. These logs record transaction hashes, gas consumption estimates, and event emissions (Transfer), (onlyShopOwner).
- 4. Environment Configuration & Deployment Setup:** Step-by-step configuration guidelines for setting up the Hardhat local EVM node environment (<http://127.0.0.1:8545/>). It outlines network configuration parameters, private key imports, and JSON-RPC provider settings required to mirror production deployment conditions.
- 5. Security & Regulatory Considerations:** An analysis of non-custodial wallet security, data privacy, and smart contract safety measures. This section details how customer privacy is preserved by mapping phone numbers to wallet addresses without storing sensitive personal data on-chain, ensuring compliance with decentralized data protection principles.

The appendix enriches the main project report by providing a holistic view of the technical architecture, execution logs, interface design, and security compliance of BlockLoyalty, demonstrating its viability as a modern, blockchain-based retail loyalty platform.

## **Acknowledgements**

I am profoundly grateful to Prof. Manila Gupta for her expert guidance and continuous encouragement throughout to see that this project reaches its target.

I would like to express deepest appreciation towards Dr. Varsha Shah, Principal RCOE, Mumbai and Prof. Shiburaj Pappu, Dean (Academics) whose invaluable guidance supported me in this project.

At last, I must express my sincere heartfelt gratitude to all the staff members of the Computer Engineering Department who helped directly or indirectly during this course of work.

Khan Zaara Nasim  
Anshika Shukla  
Sakshi Jha